

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №1
по курсу «Алгоритмы и структуры данных»
Тема: Сортировка вставками, выбором, пузырьковая
Вариант 7 (Продвинутый уровень)

Выполнила:
Кулакова Елизавета Алексеевна
К3260

Проверил:
Харлунин А. А.

Санкт-Петербург
2025 г.

Содержание отчета

Содержание отчета.....	2
Задачи по варианту.....	3
Задача №1. Сортировка вставкой.....	3
Задача №2. Сортировка вставкой+.....	8
Задача №3. Сортировка вставкой по убыванию.....	14
Сортировка вставкой по убыванию через рекурсию.....	18
Задача №4. Линейный поиск.....	24
Задача №10. Палиндром.....	30
Вывод.....	41

Задачи по варианту

Задача №1. Сортировка вставкой

Используя код процедуры Insertion-sort, напишите программу и проверьте сортировку массива $A = \{31, 41, 59, 26, 41, 58\}$.

- Формат входного файла (input.txt). В первой строке входного файла содержится число n ($1 \leq n \leq 10^3$) — число элементов в массиве. Во второй строке находятся n различных целых чисел, по модулю не превосходящих 109.

- Формат выходного файла (output.txt). Одна строка выходного файла с отсортированным массивом. Между любыми двумя числами должен стоять ровно один пробел.

- Ограничение по времени. 2сек.

- Ограничение по памяти. 256 мб.

Выберите любой набор данных, подходящих по формату, и протестируйте алгоритм.

Листинг кода.

```
from measurements import *

@measure_memory_usage
@timeit
def sort_insert():
    with open('input.txt') as f:
        data = f.readlines()

        n, arr = int(data[0]), list(map(int,
data[1].split(',')))

        for i in range(1, n):
            j = i
            while j > 0 and arr[j] < arr[j - 1]:
                arr[j - 1], arr[j] = arr[j], arr[j - 1]
                j -= 1

        with open('output.txt', 'w') as f:
            f.write(" ".join(map(str, arr)))

        return arr

if __name__ == "__main__":
    for i in (1, 10**3, None):
        generate_array(i)
```

```
sort_insert()
```

```
from measurements import *

@measure_memory_usage
@timeit
def sort_insert():
    with open('input.txt') as f:
        data = f.readlines()

    n, arr = int(data[0]), list(map(int, data[1].split(',')))

    for i in range(1, n):
        while i > 0:
            if arr[i] < arr[i-1]:
                arr[i-1], arr[i] = arr[i], arr[i-1]
            i -= 1

    with open('output.txt', 'w') as f:
        f.write(" ".join(map(str, arr)))

    return arr

if __name__ == "__main__":
    for i in (1, 10**3, None):
        generate_array(i)
        sort_insert()
```

```
import random
import numpy as np

def generate_array(n=None):
    if not n:
        n = random.randint(1, 10**3)
    arr = np.random.randint(-10**9, 10**9, size=n)

    with open('input.txt', 'w') as f:
        f.writelines([str(n), ", ".join(map(str, arr))])

    return None
```

Текстовое объяснение решения.

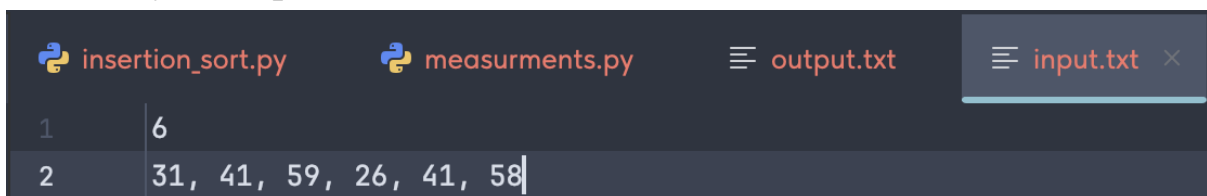
Для выполнения первого задания была написана функция `sort_insert()`, которая не принимает аргументов, но считывает данные из файла `input.txt` по заданному формату. Функция возвращает отсортированный массив чисел.

В процессе работы функции производится проход по массиву через индексы элементов от 1 до последнего ($n-1$). Производится сравнение текущего элемента с предыдущим, и, если текущий меньше предыдущего, они меняются местами. Такое сравнение производится вплоть до второго элемента массива через операцию понижения значения индекса на 1, чтобы отсортировать и часть массива, которая стоит до предыдущего элемента.

После сортировки массива в файл `output.txt` записывается строковое значение, образованное конкатенацией значений отсортированного массива. Также сам массив возвращается функцией.

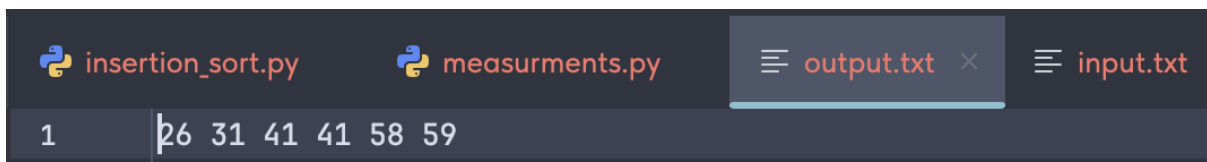
Для генерации массива была написана функция `generate_array()`, которая в качестве аргумента может принимать целое число – размерность массива. Если число при вызове не передается, то генерируется число в заданном диапазоне, а затем на его основании генерируется массив, вся информация записывается в файл `input.txt`.

Результат работы кода:



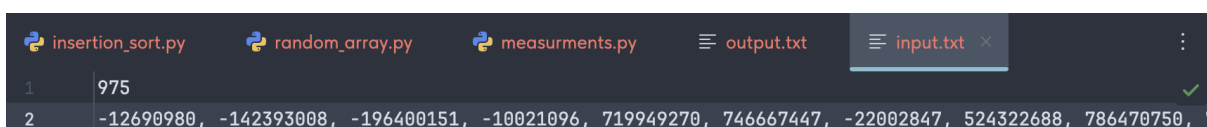
```
insertion_sort.py  measurments.py  output.txt  input.txt x
1      6
2      31, 41, 59, 26, 41, 58
```

Рисунок 1 – Файл `input.txt` с входными данными из задания



```
insertion_sort.py  measurments.py  output.txt x  input.txt
1      26 31 41 41 58 59
```

Рисунок 2 – Результат работы функции `sort_insert()` на примере из задания



```
insertion_sort.py  random_array.py  measurments.py  output.txt  input.txt x
1      975
2      -12690980, -142393008, -196400151, -10021096, 719949270, 746667447, -22002847, 524322688, 786470750, 1
```

Рисунок 3 – Файл `input.txt` со сгенерированными входными данными

Рисунок 4 – Результат работы функции sort_insert() на сгенерированном примере

Результат работы кода на максимальных и минимальных значениях:

Рисунок 5 – Файл input.txt с вводными данными – минимальными значениями

Рисунок 6 – Файл output.txt после выполнения функции sort_insert() с минимальными значениями

Рисунок 7 – Файл input.txt с вводными данными – максимальными значениями

Рисунок 8 – Файл output.txt после выполнения функции sort_insert() с максисмальными значениями

Таблица затрат памяти и времени по заданию 1:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0009 сек	234 байт
Пример из задачи	0.0037 сек	302 байт
Собственный пример	1.2436 сек	35.7 Кбайт
Верхняя граница диапазона значений входных данных из текста задачи	1.8457 сек	36.2 Кбайт

Вывод по задаче: В этой задаче был реализован алгоритм сортировки вставками. Сложность сортировки вставками в худшем и среднем случае составляет $O(n^2)$, а в лучшем случае – $O(n)$.

Задача №2. Сортировка вставкой+

Измените процедуру Insertion-sort для сортировки таким образом, чтобы в выходном файле отображалось в первой строке n чисел, которые обозначают новый индекс элемента массива после обработки.

• **Формат выходного файла (output.txt).** В первой строке выходного файла выведите n чисел. При этом i -ое число равно индексу, на который, в момент обработки его сортировкой вставками, был перемещен i -ый элемент исходного массива. Индексы нумеруются, начиная с единицы. Между любыми двумя числами должен стоять ровно один пробел.

Пример.

input.txt	output.txt
10	1 2 2 2 3 5 5 6 9 1
1 8 4 2 3 7 5 6 9 0	0 1 2 3 4 5 6 7 8 9

В примере сортировка вставками работает следующим образом:

- Первый элемент остается на своем месте, поэтому первое число в ответе – единица. Отсортированная часть массива: [1]
- Второй элемент больше первого, поэтому он тоже остается на своем месте, и второе число в ответе — двойка. [1 8]
- Четверка меньше восьмерки, поэтому занимает второе место. [1 4 8]
- Двойка занимает второе место. [1 2 4 8]
- Тройка занимает третье место. [1 2 3 4 8]
- Семерка занимает пятое место. [1 2 3 4 7 8]
- Пятерка занимает пятое место. [1 2 3 4 5 7 8]
- Шестерка занимает шестое место. [1 2 3 4 5 6 7 8]
- Девятка занимает девятое место. [1 2 3 4 5 6 7 8 9]
- Ноль занимает первое место. [0 1 2 3 4 5 6 7 8 9]

Листинг кода.

```
from measurments import *
from random_array import *

@measure_memory_usage
@timeit
def sort_insert():
    with open('input.txt') as f:
        data = f.readlines()
```

```

        n, arr = int(data[0]), list(map(int,
data[1].split(',')))

    index_array = [1]
    for i in range(1, n):
        j = i
        while j > 0 and arr[j] < arr[j - 1]:
            arr[j - 1], arr[j] = arr[j], arr[j - 1]
            j -= 1
        index_array.append(j+1)

    with open('output.txt', 'w') as f:
        f.writelines([f'{" ".join(map(str,
index_array))}\n', " ".join(map(str, arr))])

    return arr

if __name__ == "__main__":
    sort_insert()

    for i in (1, 10**3, None):
        generate_array(i)
        sort_insert()

```

```

from measurments import *
from random_array import *

```

```
@measure_memory_usage
```

```
@timeit
```

```
def sort_insert():
```

```
    with open('input.txt') as f:
```

```
        data = f.readlines()
```

```
    n, arr = int(data[0]), list(map(int, data[1].split(',')))
```

```
    index_array = [1]
```

```
    for i in range(1, n):
```

```
        j = i
```

```
        while j > 0 and arr[j] < arr[j - 1]:
```

```
            arr[j - 1], arr[j] = arr[j], arr[j - 1]
```

```
            j -= 1
```

```
        index_array.append(j+1)
```

```

with open('output.txt', 'w') as f:
    f.writelines([f{" ".join(map(str, index_array))}\n', " ".join(map(str, arr))])

return arr

if __name__ == "__main__":
    sort_insert()

for i in (1, 10**3, None):
    generate_array(i)
    sort_insert()

```

Текстовое объяснение решения.

Для выполнения второго задания была использована функция `sort_insert()` из задания 1, которая не принимает аргументов, но считывает данные из файла `input.txt` по заданному формату. Функция возвращает отсортированный массив чисел.

В процессе работы функции производится проход по массиву через индексы элементов от 1 до последнего ($n-1$). Для фиксации индексов в процессе сортировки создается массив `index_arr` с 1, так как первый элемент сам по себе уже отсортирован. Производится сравнение текущего элемента с предыдущим, и, если текущий меньше предыдущего, они меняются местами. Такое сравнение производится вплоть до второго элемента массива через операцию понижения значения индекса на 1, чтобы отсортировать и часть массива, которая стоит до предыдущего элемента. Итоговый индекс текущего элемента в новом массиве добавляется в массив `index_arr`.

После сортировки массива в файл `output.txt` записываются два строковых значения: образованное конкатенацией значений массива индексов и образованное конкатенацией значений отсортированного массива. Также сам массив возвращается функцией.

Для генерации массива была написана функция `generate_array()`, которая в качестве аргумента может принимать целое число – размерность массива. Если число при вызове не передается, то генерируется число в заданном диапазоне, а затем на его основании генерируется массив, вся информация записывается в файл `input.txt`.

Результат работы кода на примерах из задания:

```
insertion_sort.py  input.txt  output.txt
1 | 6
2 | 31, 41, 59, 26, 41, 58
```

Рисунок 9 – Файл input.txt с входными данными из задания 1

```
insertion_sort.py  input.txt  output.txt
1 | 1 2 3 1 4 5
2 | 26 31 41 41 58 59
```

Рисунок 10 – Результат работы кода для данных из задания 1, записанный в output.txt

```
insertion_sort.py  input.txt  output.txt
1 | 10
2 | 1, 8, 4, 2, 3, 7, 5, 6, 9, 0
```

Рисунок 11 – Файл input.txt с входными данными из задания 2

```
insertion_sort.py  input.txt  output.txt
1 | 1 2 2 2 3 5 5 6 9 1
2 | 0 1 2 3 4 5 6 7 8 9
```

Рисунок 12 – Результат работы кода для данных из задания 2, записанный в output.txt

```
insertion_sort.py  input.txt  output.txt  insertion_sort_desc.py
1 | 685
2 | 995602509, -142089620, -810933719, -127199006, 410690251, -971337400, 913316522, 837434710, -304787076
```

Рисунок 13 – Файл input.txt со сгенерированными входными данными

```
insertion_sort.py  input.txt  output.txt  insertion_sort_desc.py
1 | 1 1 1 3 4 1 6 6 3 6 8 3 2 5 13 6 3 8 12 11 9 9 19 16 11 17 6 7 13 29 21 3 25 17 30 27 29 17 24 19 1
2 | -993292949 -993093269 -991236451 -990536370 -989788048 -989269418 -985709632 -983238236 -982022537 -981
```

Рисунок 14 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:

```

insertion_sort.py  input.txt  output.txt
1 | 1
2 | -693982919

```

Рисунок 15 – Минимальное вводимое значение в input.txt

```

insertion_sort.py  input.txt  output.txt
1 | 1
2 | -693982919

```

Рисунок 16 – Результат работы кода для значения $n = 1$, записанный в output.txt

```

insertion_sort.py  input.txt  output.txt  insertion_sort_desc.py
1 | 1000
2 | -186773030, 440011921, -298575619, -238054153, -880007258, 126038720, -119065106, 603969050, -1116654

```

Рисунок 17 – Максимальное вводимое значение в input.txt

```

insertion_sort.py  input.txt  output.txt  insertion_sort_desc.py
1 | 1 2 1 2 1 5 5 8 6 6 11 2 10 11 3 4 4 14 19 9 18 8 21 16 15 6 2 16 17 1 10 30 7 19 10 26 8 29 34 38
2 | -999745311 -995699156 -994108532 -990337919 -988081543 -985484220 -982358989 -981326720 -980506769 -9

```

Рисунок 18 – Результат работы кода для значения $n = 1000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 2:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0057 сек	234 байт
Пример из задачи 1	0.0021 сек	238 байт
Собственный пример	0.6951 сек	24.8 Кбайт
Пример из задачи 2	0.0011 сек	302 байт

Верхняя граница диапазона значений входных данных из текста задачи	0.5846 сек	36.1 Кбайт
---	------------	------------

Вывод по задаче: В задании к функции сортировки вставками был добавлен алгоритм фиксации нового индекса текущего элемента после итерации.

Задача №3. Сортировка вставкой по убыванию

Перепишите процедуру Insertion-sort для сортировки в невозрастающем порядке вместо неубывающего с использованием процедуры Swap.

Формат входного и выходного файла и ограничения - как в задаче 1.

Подумайте, можно ли переписать алгоритм сортировки вставкой с использованием рекурсии?

Листинг кода.

```
from measurments import *
from random_array import *

@measure_memory_usage
@timeit
def sort_desc_insert():
    with open('input.txt') as f:
        data = f.readlines()

        n, arr = int(data[0]), list(map(int,
data[1].split(',')))

        if n > 1:
            for i in range(1, n):
                current = arr[i]
                pos = i

                while pos > 0 and arr[pos - 1] < current:
                    arr[pos], arr[pos - 1] = arr[pos - 1],
arr[pos]
                    pos -= 1

            with open('output.txt', 'w') as f:
                f.write(" ".join(map(str, arr)))
            return arr

if __name__ == "__main__":
    generate_array(1000)
    with open('input.txt') as f:
        inp = f.readlines()
    array = list(map(int, inp[1].split(',')))
    my_arr = sort_desc_insert()
    print(my_arr)
    assert sorted(array, reverse=True) == my_arr, 'not
sorted!'
```

```

from measurments import *
from random_array import *

@measure_memory_usage
@timeit
def sort_desc_insert():
    with open('input.txt') as f:
        data = f.readlines()

    n, arr = int(data[0]), list(map(int, data[1].split(',')))

    if n > 1:
        for i in range(1, n):
            current = arr[i]
            pos = i

            while pos > 0 and arr[pos - 1] < current:
                arr[pos], arr[pos - 1] = arr[pos - 1], arr[pos]
                pos -= 1

        with open('output.txt', 'w') as f:
            f.write(" ".join(map(str, arr)))
        return arr

if __name__ == "__main__":
    sort_desc_insert()

```

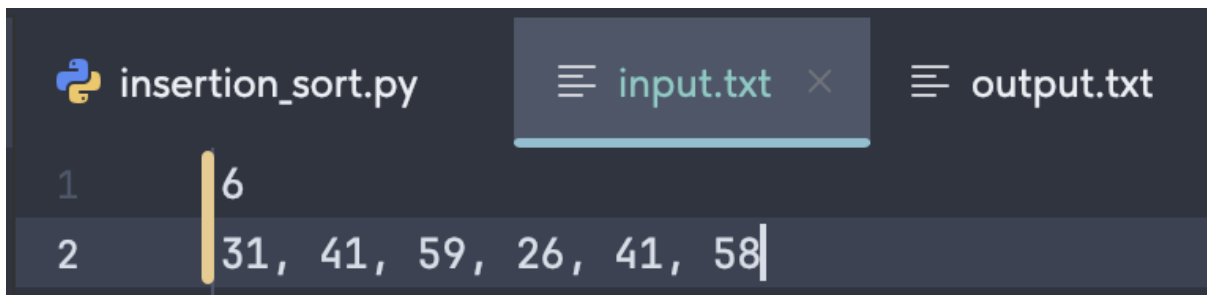
Текстовое объяснение решения.

Длина массива и сам массив считываются из файла input.txt. Если длина массива равна 1, то сортировка не выполняется и возвращается тот же массив, что и был считан.

Если длина массива больше 1, то выполняется сортировка. Фиксируем исследуемый элемент и его позицию для дальнейшего сдвига влево. Пока элемент справа больше элемента слева и позиция, которую сдвигаем, больше 0, меняем элементы местами через swap. Такую процедуру проделываем для каждого элемента массива. Фактически

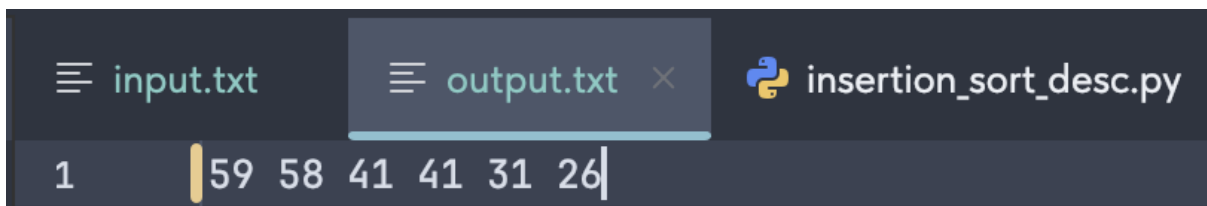
выполняется сортировка меньшей части массива. Отсортированный массив записывается в файл output.txt.

Результат работы кода на примерах из задания:



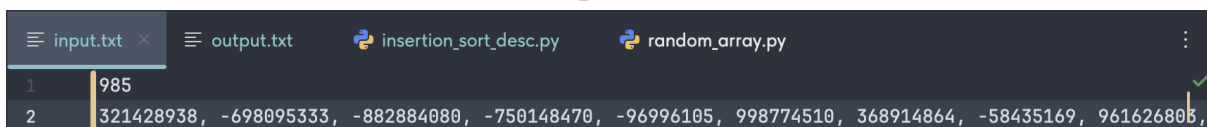
```
insertion_sort.py
1 6
2 31, 41, 59, 26, 41, 58
```

Рисунок 19 – Файл input.txt с входными данными из задания 1



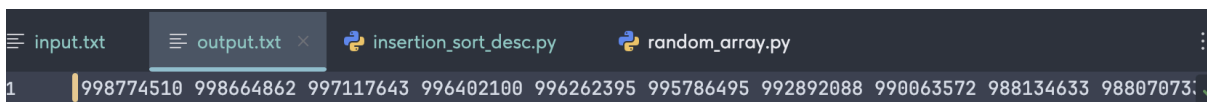
```
insertion_sort_desc.py
1 59 58 41 41 31 26
```

Рисунок 20 – Результат работы кода для данных из задания 1, записанный в output.txt



```
random_array.py
1 985
2 321428938, -698095333, -882884080, -750148470, -96996105, 998774510, 368914864, -58435169, 961626805,
```

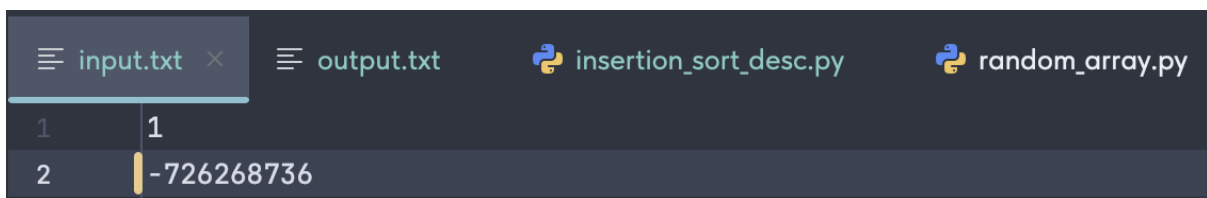
Рисунок 21 – Файл input.txt со сгенерированными входными данными



```
random_array.py
1 998774510 998664862 997117643 996402100 996262395 995786495 992892088 990063572 988134633 98807073,
```

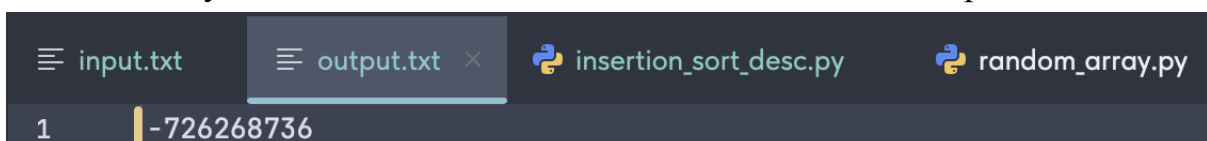
Рисунок 22 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:



```
random_array.py
1 1
2 -726268736
```

Рисунок 23 – Минимальное вводимое значение в input.txt



```
random_array.py
1 -726268736
```

Рисунок 24 – Результат работы кода для значения $n = 1$, записанный в output.txt

```

input.txt  output.txt  insertion_sort_desc.py  random_array.py
1 1000
2 412218390, 804591871, -998089575, 876377680, -35293402, 129967636, 207174059, 663089988, 408821022, 4:
  
```

Рисунок 25 – Максимальное вводимое значение в input.txt

```

input.txt  output.txt  insertion_sort_desc.py  random_array.py
1 998519014 998008175 990029923 989591615 982226410 977918566 973912834 973670941 970264281 97014405'
  
```

Рисунок 26 – Результат работы кода для значения $n = 1000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 3:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0007 сек	234 байт
Пример из задачи 1	0.0011 сек	238 байт
Собственный пример	0.5685 сек	35.7 Кбайт
Верхняя граница диапазона значений входных данных из текста задачи	0.5971 сек	36.1 Кбайт

Вывод по задаче: Написан и изучен алгоритм сортировки вставками по убыванию.

Сортировка вставкой по убыванию через рекурсию

Подумайте, можно ли переписать алгоритм сортировки вставкой с использованием рекурсии?

Листинг кода

```
from measurments import *
from random_array import *

@measure_memory_usage
@timeit
def recursion_insert():
    with open('input.txt') as f:
        data = f.readlines()

        n, init_arr = int(data[0]), list(map(int,
data[1].split(',')))

    if n <= 1:
        return init_arr

    k = init_arr[-1]
    with open('input.txt', 'w') as f:
        f.write(f'{str(n-1)}\n')
        f.write(", ".join(map(str, init_arr[:-1])))
    arr = recursion_insert()
    arr.append(k)

    for i in range(1, n):
        current = arr[i]
        pos = i

        while pos > 0 and arr[pos - 1] < current:
            arr[pos], arr[pos - 1] = arr[pos - 1],
arr[pos]
            pos -= 1

    with open('output.txt', 'w') as f:
        f.write(" ".join(map(str, arr)))
    return arr

if __name__ == "__main__":
    generate_array(6)
    with open('input.txt') as fl:
        inp = fl.readlines()
```

```

    ln, array = inp[0], list(map(int, inp[1].split(',')))

    sort_arr = recursion_insert()
    print(f'INITIAL    ARRAY:    {array},\nSORTED    ARRAY:
{sort_arr}')
    with open('input.txt', 'w') as fl:
        fl.writelines([f'{ln}', f'{"', '".join(map(str,
array))}''])
    assert sorted(array, reverse=True) == sort_arr, 'not
sorted!!!!'

```

```

from measurments import *
from random_array import *

```

```
@measure_memory_usage
```

```
@timeit
```

```
def recursion_insert():
```

```
    with open('input.txt') as f:
```

```
        data = f.readlines()
```

```
    n, init_arr = int(data[0]), list(map(int, data[1].split(',')))
```

```
    if n <= 1:
```

```
        return init_arr
```

```
    k = init_arr[-1]
```

```
    with open('input.txt', 'w') as f:
```

```
        f.write(f'{str(n-1)}\n')
```

```
        f.write(", ".join(map(str, init_arr[:-1])))
```

```
    arr = recursion_insert()
```

```
    arr.append(k)
```

```
    for i in range(1, n):
```

```
        current = arr[i]
```

```
        pos = i
```

```
        while pos > 0 and arr[pos - 1] < current:
```

```
            arr[pos], arr[pos - 1] = arr[pos - 1], arr[pos]
```

```
            pos -= 1
```

```

with open('output.txt', 'w') as f:
    f.write(" ".join(map(str, arr)))
return arr

if __name__ == "__main__":
    generate_array()
    with open('input.txt') as fl:
        inp = fl.readlines()

    ln, array = inp[0], list(map(int, inp[1].split(',')))

    sort_arr = recursion_insert()
    print(f'INITIAL ARRAY: {array},\nSORTED ARRAY: {sort_arr}')

    with open('input.txt', 'w') as fl:
        fl.writelines([f'{ln}', f'{", ".join(map(str, array))}'])

    assert sorted(array, reverse=True) == sort_arr, 'not sorted!!!!'

```

Листинг кода нового декоратора для рекурсии

```

import time
import tracemalloc
from functools import wraps

# глобальные переменные, чтобы понимать, что мы
# входим/выходим из ВНЕШНЕЙ рекурсии
_in_recursion = False
_start_time = None

def measure_performance(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        global _in_recursion, _start_time

        is_outer_call = not _in_recursion
        if is_outer_call:
            # запускаем мониторинг только при первом входе
            # в рекурсию
            _in_recursion = True
            _start_time = time.perf_counter()
            tracemalloc.start()

```

```

        # выполняем рекурсивную функцию
        result = func(*args, **kwargs)

        if is_outer_call:
            # замеряем время и память только при выходе из
            # внешнего уровня
            end_time = time.perf_counter()
            total_time = end_time - _start_time

            current, peak =
            tracemalloc.get_traced_memory()
            tracemalloc.stop()

            print(f"Function {func.__name__} took
            {total_time:.4f} seconds")
            print(f"Peak memory usage: {peak / 1024:.2f}
            KB")

            print("-" * 50)

            # сбрасываем флаг
            _in_recursion = False

        return result

    return wrapper

```

Тестовое объяснение решения

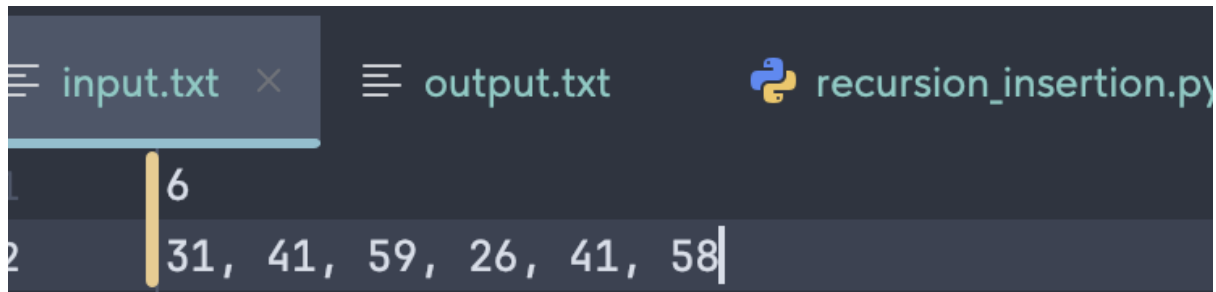
В сортировке вставкой по убыванию алгоритм рекурсии был реализован следующим образом: на вход из файла input.txt подаются длина массива и массив. Фиксируется последний элемент массива в переменной k. В файл input.txt записывается n-1 и изначальный массив без последнего элемента, чтобы отсортировать массив рекурсивно (сначала длины 1, потом длины 2 и так до длины n-1). Когда массив меньшей длины отсортирован, к нему добавляется последний элемент из переменной k. Происходит сортировка массива длины на 1 больше, чем был возвращен функцией recursion_insert().

Сортировка производится с помощью уже написанной функции sort_desc_insert(). Отсортированный массив записывается в файл output.txt.

Опытным путем было выяснено, что ошибка рекурсии не достигается до значения n=499 не включительно.

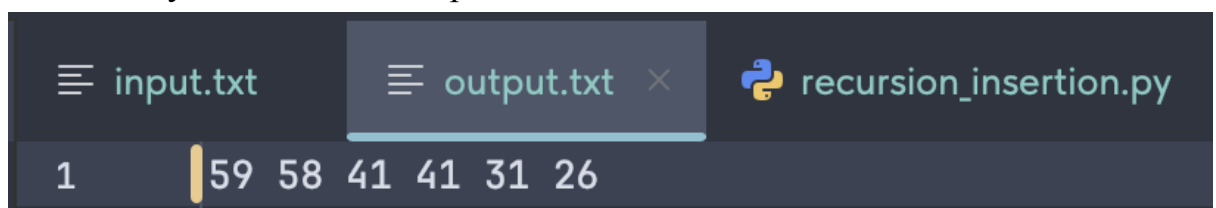
Также было замечено, что при классических замерах памяти функция сильно замедляется (при значении 332 время выполнения было 661 сек), соответственно, был скорректирован и декоратор.

Результат работы кода на примерах из задания:



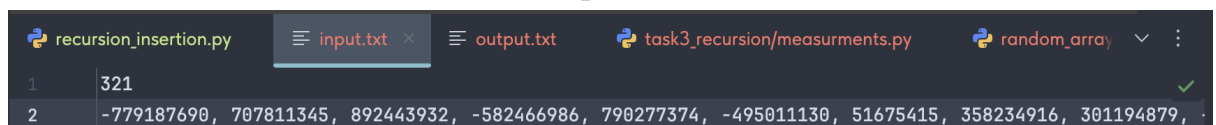
```
input.txt  output.txt  recursion_insertion.py
6
31, 41, 59, 26, 41, 58
```

Рисунок 27 – Файл input.txt с входными данными из задания 1



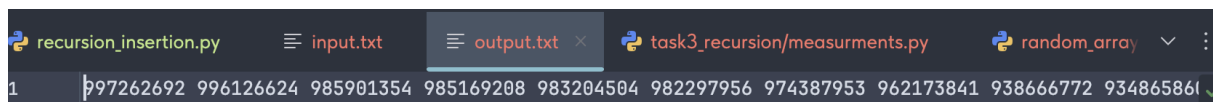
```
input.txt  output.txt  recursion_insertion.py
1 59 58 41 41 31 26
```

Рисунок 28 – Результат работы кода для данных из задания 1, записанный в output.txt



```
recursion_insertion.py  input.txt  output.txt  task3_recursion/measurements.py  random_array
1 321
2 -779187690, 707811345, 892443932, -582466986, 790277374, -495011130, 51675415, 358234916, 301194879,
```

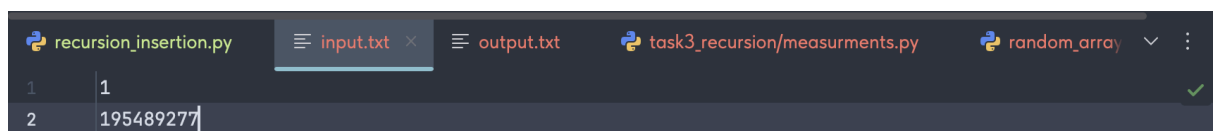
Рисунок 29 – Файл input.txt со сгенерированными входными данными



```
recursion_insertion.py  input.txt  output.txt  task3_recursion/measurements.py  random_array
1 97262692 996126624 985901354 985169208 983204504 982297956 974387953 962173841 938666772 934865861
```

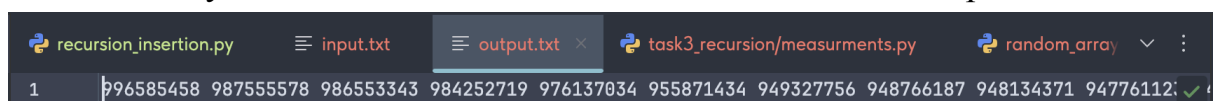
Рисунок 30 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:



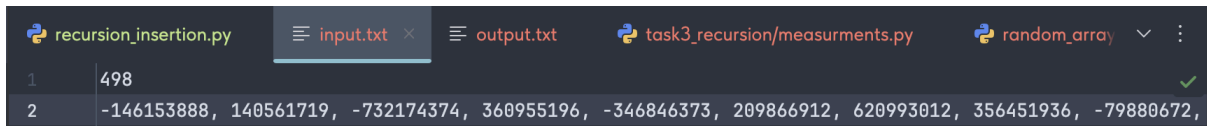
```
recursion_insertion.py  input.txt  output.txt  task3_recursion/measurements.py  random_array
1 1
2 195489277
```

Рисунок 31 – Минимальное вводимое значение в input.txt



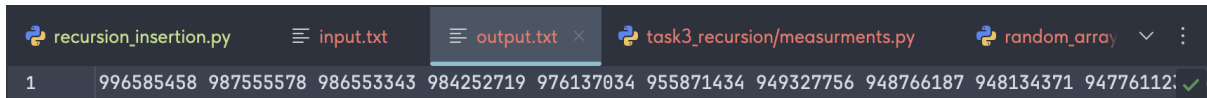
```
recursion_insertion.py  input.txt  output.txt  task3_recursion/measurements.py  random_array
1 96585458 987555578 986553343 984252719 976137034 955871434 949327756 948766187 948134371 94776112
```

Рисунок 32 – Результат работы кода для значения $n = 1$, записанный в output.txt



```
recursion_insertion.py  input.txt  output.txt  task3_recursion/measurments.py  random_array  ⌵  ⋮  
1 498  
2 -146153888, 140561719, -732174374, 360955196, -346846373, 209866912, 620993012, 356451936, -79880672,
```

Рисунок 33 – Максимальное вводимое значение в input.txt



```
recursion_insertion.py  input.txt  output.txt  task3_recursion/measurments.py  random_array  ⌵  ⋮  
1 996585458 987555578 986553343 984252719 976137034 955871434 949327756 948766187 948134371 94776112: ✓
```

Рисунок 34 – Результат работы кода для значения $n = 498$, записанный в output.txt

Таблица затрат памяти и времени по заданию 3_рекурсия:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0004 сек	13.67 Кбайт
Пример из задачи 1	0.0062 сек	21.58 Кбайт
Собственный пример	1.1772 сек	2902.34 Кбайт
Верхняя граница диапазона значений входных данных из текста задачи	1.2633 сек	3083.81 Кбайт

Вывод по задаче: Сортировку с помощью рекурсии реализовать вполне реально, причем вписываясь в ограничения. Конечно, рекурсия будет занимать больше памяти по объему, чем итеративный алгоритм, в силу самовывозов.

Задача №4. Линейный поиск

Рассмотрим задачу поиска.

- Формат входного файла. Последовательность из n чисел $A = a_1, a_2, \dots, a_n$ в первой строке, числа разделены пробелом, и значение V во второй строке. Ограничения: $0 \leq n \leq 10^3$, $-10^3 \leq a_i$, $V \leq 10^3$
- Формат выходного файла. Одно число - индекс i , такой, что $V = A[i]$, или значение -1 , если V в отсутствует.
- Напишите код линейного поиска, при работе которого выполняется сканирование последовательности в поисках значения V .
- Если число встречается несколько раз, то выведите, сколько раз встречается число и все индексы i через запятую.
- Дополнительно: попробуйте найти свинью, как в лекции. Используйте во входном файле последовательность слов из лекции, и найдите соответствующий индекс.

Листинг кода.

```
from measurments import *
from random_array import *

@measure_memory_usage
@timeit
def linear_search():
    with open('input.txt') as f:
        data = f.readlines()

    arr, v = list(map(str, data[0].split())), str(data[1])

    v_inds = []

    if len(arr) < 2:
        if len(arr) == 0:
            v_inds = []
        else:
            if arr[0] == v:
                v_inds.append(0)
            else:
                v_inds = []
    else:
        for i in range(len(arr)):
            if v == arr[i]:
                v_inds.append(i)
```

```

        output = f'{"", ".join(map(str, v_inds))}'.strip(', ')
    if v_inds else str(-1)
    with open('output.txt', 'w') as f:
        if len(v_inds) > 1:
            f.writelines([f'{len(v_inds)}\n', output])
        else:
            f.write(output)
    return output

if __name__ == "__main__":
    generate_array()
    print(linear_search())

```

```

from measurments import *
from random_array import *

```

```
@measure_memory_usage
```

```
@timeit
```

```
def linear_search():
```

```

    with open('input.txt') as f:
        data = f.readlines()

```

```
arr, v = list(map(str, data[0].split())), str(data[1])
```

```
v_inds = []
```

```
if len(arr) < 2:
```

```
    if len(arr) == 0:
```

```
        v_inds = []
```

```
    else:
```

```
        if arr[0] == v:
```

```
            v_inds.append(0)
```

```
        else:
```

```
            v_inds = []
```

```
else:
```

```
    for i in range(len(arr)):
```

```
        if v == arr[i]:
```

```
            v_inds.append(i)
```

```
output = f'{"", ".join(map(str, v_inds))}'.strip(', ') if v_inds else str(-1)
```

```

with open('output.txt', 'w') as f:
    if len(v_inds) > 1:
        f.writelines([f'{len(v_inds)}\n', output])
    else:
        f.write(output)
return output

if __name__ == "__main__":
    generate_array()
    print(linear_search())

```

Листинг кода для дополнительной функции

```

import random
import numpy as np

def generate_array(n=None):
    if n not in range(0, 10**3 + 1):
        n = random.randint(0, 10**3)
    arr = np.random.randint(-10**3, 10**3, size=n)
    v = np.random.randint(-10**3, 10**3)

    with open('input.txt', 'w') as f:
        f.writelines([f'{" ".join(map(str, arr))}\n',
str(v)])

    return None

```

```

import random
import numpy as np

def generate_array(n=None):
    if n not in range(0, 10**3 + 1):
        n = random.randint(0, 10**3)
    arr = np.random.randint(-10**3, 10**3, size=n)
    v = np.random.randint(-10**3, 10**3)

    with open('input.txt', 'w') as f:
        f.writelines([f'{" ".join(map(str, arr))}\n', str(v)])

    return None

```

Текстовое объяснение решения.

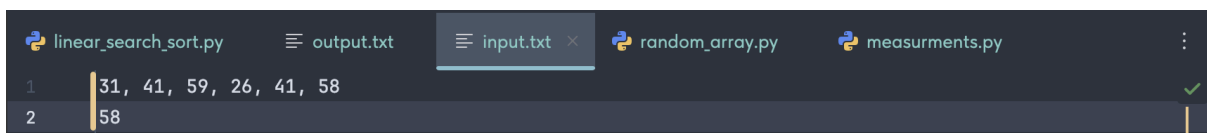
Алгоритм линейного поиска реализуется довольно просто. Из файла `input.txt` считываются массив и значение, индекс которого необходимо найти в имеющемся массиве. Создается пустой список `v_inds` для записи индексов искомого значения. Сначала проверяется длина массива, и, если она меньше двух, искомый элемент сравнивается с единственным элементом (в случае длины 1) или сразу возвращается пустой список (при длине 0). Если длина массива больше 1, то начинается итерация. Через цикл `for` для каждого индекса массива производится сравнение с искомым элементом. Если элемент массива по текущему индексу совпадает с искомым, его индекс записывается в `v_inds`.

Если список `v_inds` пуст, то в переменную `output` записывается значение `-1`. Иначе в `output` записывается строка, составленная из элементов массива.

Если элементов в массиве больше 1, то в файл `output.txt` записывается количество найденных индексов и все индексы через запятую. Иначе записывается единственное значение переменной `output`.

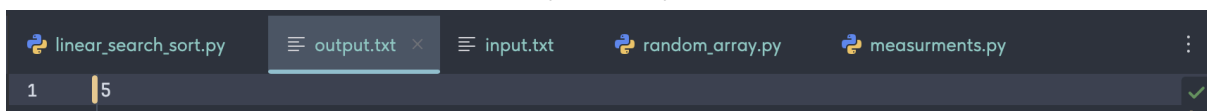
Для генерации массивов для формирования примеров была написана функция `generate_array()`.

Результат работы кода на примерах из задания:



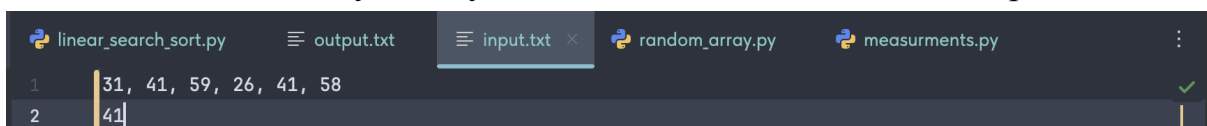
```
linear_search_sort.py  output.txt  input.txt  random_array.py  measurments.py
1 31, 41, 59, 26, 41, 58
2 58
```

Рисунок 35 – Файл `input.txt` с входными данными из задания 1 + единственный существующий элемент



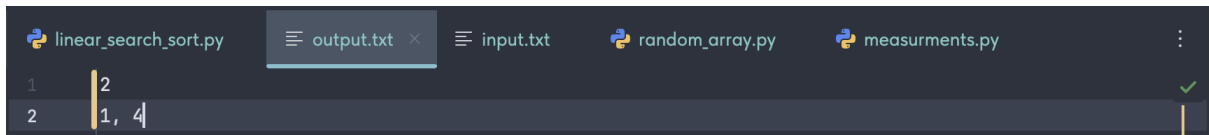
```
linear_search_sort.py  output.txt  input.txt  random_array.py  measurments.py
1 5
```

Рисунок 36 – Результат работы кода для данных из задания 1 + единственный существующий элемент, записанный в `output.txt`



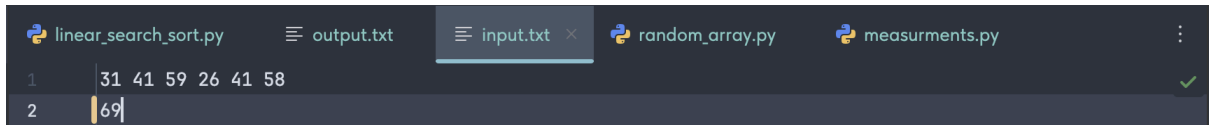
```
linear_search_sort.py  output.txt  input.txt  random_array.py  measurments.py
1 31, 41, 59, 26, 41, 58
2 41
```

Рисунок 37 – Файл `input.txt` с входными данными из задания 1 + не единственно существующий элемент



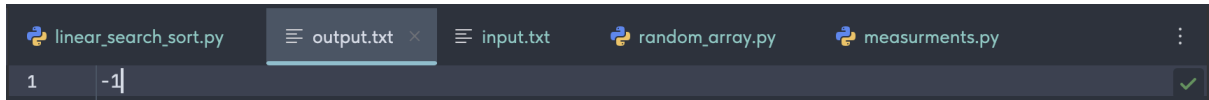
```
linear_search_sort.py  output.txt x input.txt  random_array.py  measurments.py
1 2
2 1, 4
```

Рисунок 38 – Файл input.txt с входными данными из задания 1 + не единственно существующий элемент



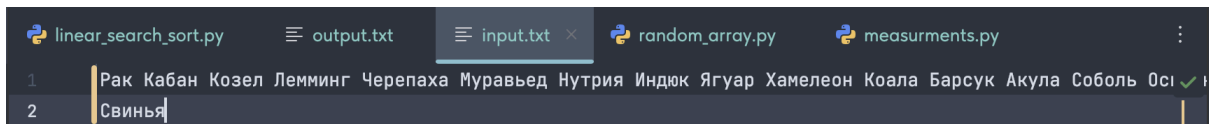
```
linear_search_sort.py  output.txt x input.txt  random_array.py  measurments.py
1 31 41 59 26 41 58
2 69
```

Рисунок 39 – Файл input.txt с входными данными из задания 1 + несуществующий элемент



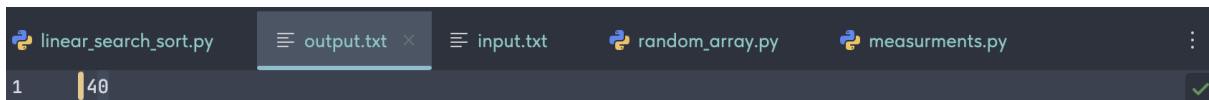
```
linear_search_sort.py  output.txt x input.txt  random_array.py  measurments.py
1 -1
2
```

Рисунок 40 – Файл input.txt с входными данными из задания 1 + несуществующий элемент



```
linear_search_sort.py  output.txt x input.txt  random_array.py  measurments.py
1 Рак Кабан Козел Лемминг Черепаха Муравьед Нутрия Индюк Ягуар Хамелеон Коала Барсук Акула Соболь Осел
2 Свинья
```

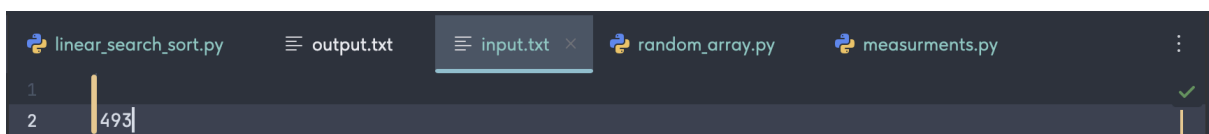
Рисунок 41 – Файл input.txt с текстовыми входными данными



```
linear_search_sort.py  output.txt x input.txt  random_array.py  measurments.py
1 40
2
```

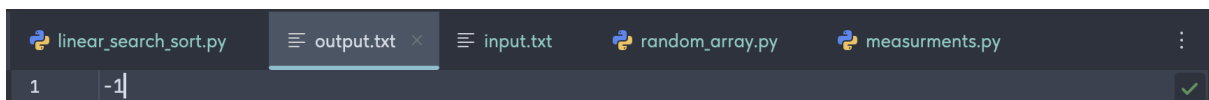
Рисунок 42 – Результат работы кода для текстовых данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:



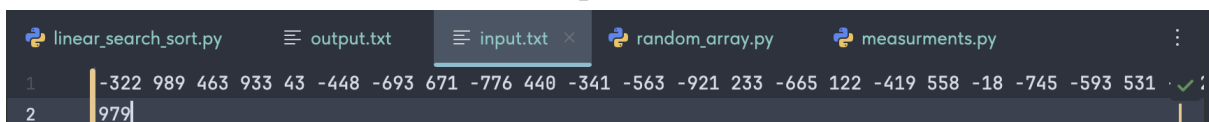
```
linear_search_sort.py  output.txt x input.txt  random_array.py  measurments.py
1
2 493
```

Рисунок 43 – Минимальное вводимое значение в input.txt



```
linear_search_sort.py  output.txt x input.txt  random_array.py  measurments.py
1 -1
2
```

Рисунок 44 – Результат работы кода для значения $n = 1$, записанный в output.txt



```
linear_search_sort.py  output.txt x input.txt  random_array.py  measurments.py
1 -322 989 463 933 43 -448 -693 671 -776 440 -341 -563 -921 233 -665 122 -419 558 -18 -745 -593 531
2 979
```

Рисунок 45 – Максимальное вводимое значение в input.txt

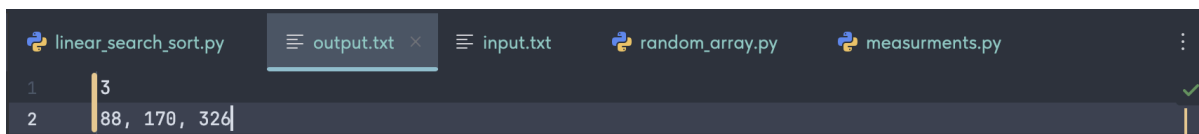


Рисунок 46 – Результат работы кода для значения $n = 1000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 4:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0006 сек	161 байт
Пример из задачи 1 + единственно существующий элемент	0.0019 сек	160 байт
Пример из задачи 1 + не единственно существующий элемент	0.0011 сек	163 байт
Пример из задачи 1 + несуществующий элемент	0.0011 сек	118 байт
Пример с текстом	0.0020 сек	161 байт
Верхняя граница диапазона значений входных данных из текста задачи	0.0022 сек	172 байт

Вывод по задаче: Алгоритм линейного поиска реализуется довольно просто и логично, но сложность его $O(n)$, что есть не самый лучший случай.

Задача №10. Палиндром

Палиндром - это строка, которая читается одинаково как справа налево, так и слева направо. На вход программы поступает набор больших латинских букв (не обязательно различных). Разрешается переставлять буквы, а также удалять некоторые буквы.

Требуется из данных букв по указанным правилам составить палиндром наибольшей длины, а если таких палиндромов несколько, то выбрать первый из них в алфавитном порядке.

- Формат входного файла (input.txt). В первой строке входных данных содержится число n ($1 \leq n \leq 100000$). Во второй строке задается последовательность из n больших латинских букв (буквы записаны без пробелов).

- Формат выходного файла (output.txt). В единственной строке выходных данных выдайте искомый палиндром.

- Пример 1:

input.txt	output.txt
5 AAB	ABA

- Пример 2:

input.txt	output.txt
6 QAZQAZ	AQZZQA

- Пример 3:

input.txt	output.txt
6 ABCDEF	A

- Ограничение по времени. 1сек.
- Ограничение по памяти. 64 мб.

Листинг кода

```
from measurements import *
from random_letters import *

def letters_sort(tuples):
    # sort in descending order by the number of letters'
    occurrences
```

```

n = len(tuples)
if n > 1:
    for i in range(1, n):
        current = tuples[i][1]
        pos = i

        while pos > 0 and tuples[pos - 1][1] < current:
            tuples[pos], tuples[pos - 1] = tuples[pos -
1], tuples[pos]
            pos -= 1

sorted_vals = []
for k, v in tuples:
    if v not in sorted_vals:
        sorted_vals.append(v)

# find letters with equal number of occurrences
for v in sorted_vals:
    v_inds = []
    for i in range(len(tuples)):
        if tuples[i][1] == v:
            v_inds.append(i)
    # if found, sort in alphabetical order
    if len(v_inds) > 1:
        temp = tuples[v_inds[0]:v_inds[-1] + 1]
        n = len(temp)
        for i in range(1, n):
            j = i
            while j > 0 and temp[j][0] < temp[j -
1][0]:
                temp[j - 1], temp[j] = temp[j], temp[j
- 1]
                j -= 1

            tuples[v_inds[0]:v_inds[-1] + 1] = temp

    return tuples

def create_palindrome(tuples):
    # check if any letters number of occurrences can make
at least 1 pair
    if any([True if (v // 2) > 0 else False for k, v in
tuples]):
        truth = [True if (v // 2) > 0 else False for k, v
in tuples]
        true_inds = []
        for i in range(len(truth)):

```

```

        if truth[i]:
            true_inds.append(i)
    right_part = ''
    left_part = ''
    for k in true_inds:
        pairs = tuples[k][1] // 2
        left_part += tuples[k][0] * pairs
        right_part = tuples[k][0] * pairs + right_part
        tuples[k] = (tuples[k][0], tuples[k][1] -
(pairs * 2))

    tuples = letters_sort(tuples)
    # check if any letters number of occurrences are
non-zero
    if any([True if v > 0 else False for k, v in
tuples]):
        truth = [True if v > 0 else False for k, v in
tuples]

        true_inds = []
        for i in range(len(truth)):
            if truth[i]:
                true_inds.append(i)
        # make in alphabetical order
        let_ind = true_inds[0]
        left_part += tuples[let_ind][0]

        return left_part + right_part
    return left_part + right_part
return tuples[0][0]

@measure_memory_usage
@timeit
def plndrm():
    with open('input.txt') as f:
        data = f.readlines()

    n, crazy_code = int(data[0]), str(data[1])
    if n < 2:
        with open('output.txt', 'w') as f:
            f.writelines(crazy_code)
        return n, crazy_code

    tuples = []
    let_set = set()
    for i in crazy_code:
        if i not in let_set:

```

```

        tuples.append((i, 1))
        let_set.add(i)
    else:
        for l in range(len(tuples)):
            if tuples[l][0] == i:
                num = tuples[l][1]
                tuples[l] = (i, num + 1)

    sorted_tup = letters_sort(tuples)
    output = create_palindrome(sorted_tup)

    with open('output.txt', 'w') as f:
        f.writelines(output)

    return len(output), output

if __name__ == "__main__":
    abracadabra()
    ln, palind = plndrm()
    if len(palind) % 2 == 0:
        lft = palind[:len(palind)//2]
        right = palind[len(palind)//2:]
        right = right[::-1]
        assert lft == right, 'not a palindrome.....'
    else:
        lft = palind[:len(palind) // 2]
        right = palind[(len(palind) // 2) + 1:]
        right = right[::-1]
        assert lft == right, 'not a palindrome.....'

    print(ln, palind)

```

```

from measurments import *

```

```

from random_letters import *

```

```

def letters_sort(tuples):

```

```

    # sort in descending order by the number of letters' occurrences

```

```

    n = len(tuples)

```

```

    if n > 1:

```

```

        for i in range(1, n):

```

```

            current = tuples[i][1]

```

```

            pos = i

```

```

            while pos > 0 and tuples[pos - 1][1] < current:

```

```

        tuples[pos], tuples[pos - 1] = tuples[pos - 1], tuples[pos]
        pos -= 1

sorted_vals = []
for k, v in tuples:
    if v not in sorted_vals:
        sorted_vals.append(v)

# find letters with equal number of occurrences
for v in sorted_vals:
    v_inds = []
    for i in range(len(tuples)):
        if tuples[i][1] == v:
            v_inds.append(i)
    # if found, sort in alphabetical order
    if len(v_inds) > 1:
        temp = tuples[v_inds[0]:v_inds[-1] + 1]
        n = len(temp)
        for i in range(1, n):
            j = i
            while j > 0 and temp[j][0] < temp[j - 1][0]:
                temp[j - 1], temp[j] = temp[j], temp[j - 1]
            j -= 1

        tuples[v_inds[0]:v_inds[-1] + 1] = temp

return tuples

def create_palindrome(tuples):
    # check if any letters number of occurrences can make at least 1 pair
    if any([True if (v // 2) > 0 else False for k, v in tuples]):
        truth = [True if (v // 2) > 0 else False for k, v in tuples]
        true_inds = []
        for i in range(len(truth)):
            if truth[i]:
                true_inds.append(i)
        right_part = ""
        left_part = ""
        for k in true_inds:

```

```

pairs = tuples[k][1] // 2
left_part += tuples[k][0] * pairs
right_part = tuples[k][0] * pairs + right_part
tuples[k] = (tuples[k][0], tuples[k][1] - (pairs * 2))

tuples = letters_sort(tuples)
# check if any letters number of occurrences are non-zero
if any([True if v > 0 else False for k, v in tuples]):
    truth = [True if v > 0 else False for k, v in tuples]
    true_inds = []
    for i in range(len(truth)):
        if truth[i]:
            true_inds.append(i)
    # make in alphabetical order
    let_ind = true_inds[0]
    left_part += tuples[let_ind][0]

    return left_part + right_part
return left_part + right_part
return tuples[0][0]

```

```

@measure_memory_usage
@timeit
def plndrm():
    with open('input.txt') as f:
        data = f.readlines()

    n, crazy_code = int(data[0]), str(data[1])
    if n < 2:
        with open('output.txt', 'w') as f:
            f.writelines(crazy_code)
        return n, crazy_code

    tuples = []
    let_set = set()
    for i in crazy_code:
        if i not in let_set:
            tuples.append((i, 1))
            let_set.add(i)

```

```

else:
    for l in range(len(tuples)):
        if tuples[l][0] == i:
            num = tuples[l][1]
            tuples[l] = (i, num + 1)

sorted_tup = letters_sort(tuples)
output = create_palindrome(sorted_tup)

with open('output.txt', 'w') as f:
    f.writelines(output)

return len(output), output

if __name__ == "__main__":
    abracadabra()
    ln, palind = plndrm()
    if len(palind) % 2 == 0:
        lft = palind[:len(palind)//2]
        right = palind[len(palind)//2:]
        right = right[::-1]
        assert lft == right, 'not a palindrome.....'
    else:
        lft = palind[:len(palind) // 2]
        right = palind[(len(palind) // 2) + 1:]
        right = right[::-1]
        assert lft == right, 'not a palindrome.....'

    print(ln, palind)

```

Листинг кода функции для формирования строки из букв

```

import random
import numpy as np

def abracadabra(n=None):
    if not n:
        n = random.randint(1, 10**5)
    let_nums = np.random.randint(ord('A'), ord('Z'),
size=n)
    string = ''

```

```

    for num in let_nums:
        string += chr(num)

    with open('input.txt', 'w') as f:
        f.writelines([f'{str(n)}\n', string])

    return None

```

```

import random
import numpy as np

def abracadabra(n=None):
    if not n:
        n = random.randint(1, 10**5)
    let_nums = np.random.randint(ord('A'), ord('Z'), size=n)
    string = ""
    for num in let_nums:
        string += chr(num)

    with open('input.txt', 'w') as f:
        f.writelines([f'{str(n)}\n', string])

    return None

```

Текстовое объяснение решения

Для создания палиндрома из набора латинских заглавных букв были написаны 4 функции: `abracadabra()`, `plndrm()`, `create_palindrome()` и `letters_sort()`.

Функция `abracadabra()` может принимать в качестве аргумента целое число – длину будущего набора букв. Если ничего не передано, то функция сама генерирует длину строки в заданном диапазоне. Далее функция генерирует массив из чисел-порядковых номеров заглавных латинских букв в кодировке Unicode длины `n`. Далее функция переводит номера букв в сами буквы, соединяет их в одну строку и записывает в файл `input.txt` второй строкой. Первой строкой файла `input.txt` является длина заданного набора.

Функция `plndrm()` читает из файла `input.txt` данные и проводит подсчет количества каждой буквы в строке, создавая массив кортежей (“имя буквы”, “количество буквы в данной строке”). также при проверке создается временное множество встретившихся букв, которое используется

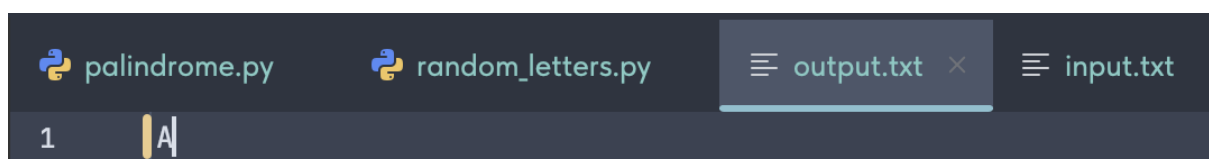
при их подсчете. Далее функция вызывает функции `letters_sort()` и `create_palindrome()` и после их выполнения возвращает длину созданного палиндрома и сам палиндром.

Функция `letters_sort()` использует алгоритм сортировки вставкой по убыванию для сортировки получившегося списка букв по убыванию количества появлений в заданной строке, а также обычную сортировку вставкой для размещения отсортированных по количеству появления букв кортежей в алфавитном порядке внутри одного значения-количества появления букв. Перед сортировкой в алфавитном порядке функция совершает линейный поиск, записывая все значения, обозначающие количество появления букв, в специальный список.

Функция `create_palindrome()` проверяет, есть ли в получившемся отсортированном массиве буквы, повторяющиеся более 1 раза, и, если такие есть, составляет из них левую и правую части палиндрома: берет целую часть от деления на 2 количества появлений буквы и получившееся количество литеры прибавляет к левой части справа и к правой части слева. Так функция проходится по каждой букве, у которой количество появлений больше 1.

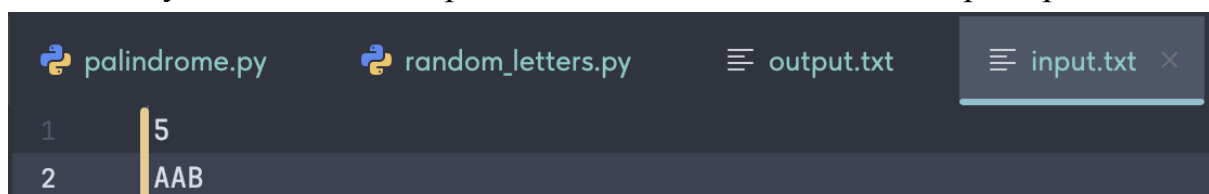
После прохождения по всем буквам с количеством больше одного функция вызывает `letters_sort()`, получает уже новый отсортированный массив, где значения у букв равны либо 1, либо 0, берет первую букву со значением 1 (если она есть) и прибавляет ее к левой части справа/соединяет уже полученные части палиндрома (если нет букв, число которых не 0) и возвращает палиндром.

Результат работы кода на примерах из задания:



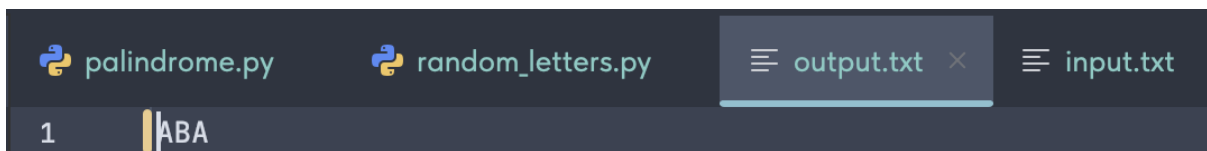
```
palindrome.py random_letters.py output.txt input.txt
1 | A
```

Рисунок 47 – Файл `input.txt` с входными данными из примера 1



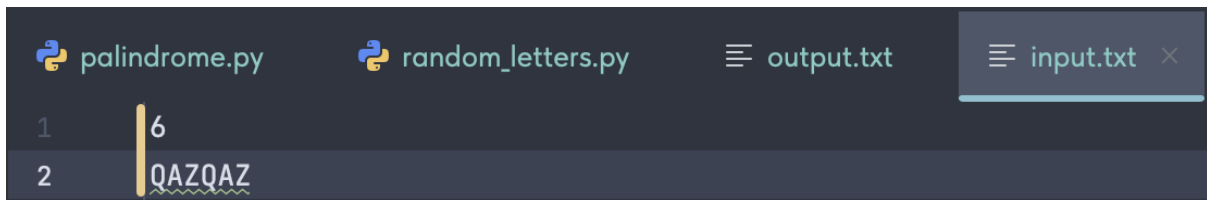
```
palindrome.py random_letters.py output.txt input.txt
1 | 5
2 | AAB
```

Рисунок 48 – Результат работы кода для данных из примера 1



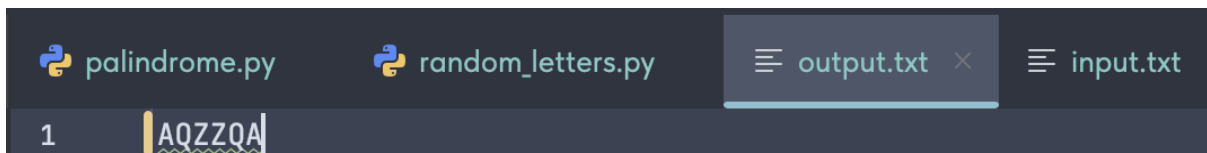
```
palindrome.py random_letters.py output.txt x input.txt
1 | ABA
```

Рисунок 49 – Файл input.txt с входными данными из примера 2



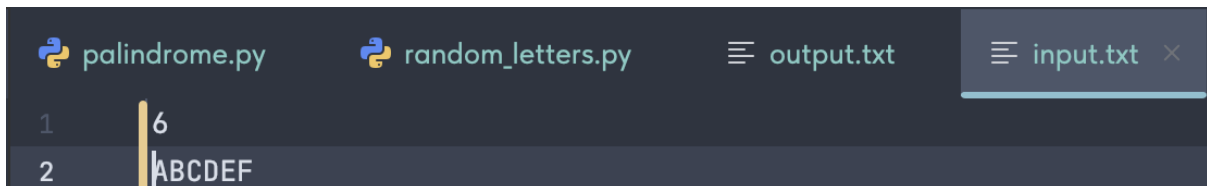
```
palindrome.py random_letters.py output.txt x input.txt
1 | 6
2 | QAZQAZ
```

Рисунок 50 – Файл output.txt с входными данными из примера 2



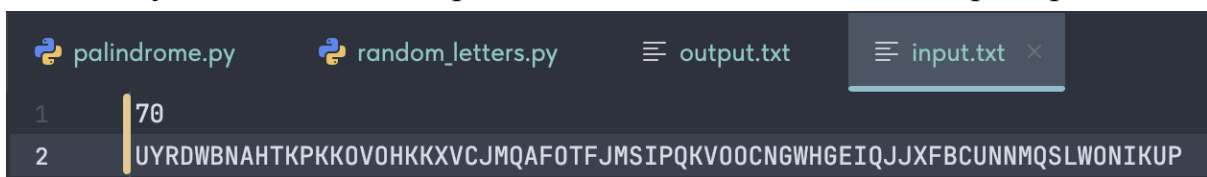
```
palindrome.py random_letters.py output.txt x input.txt
1 | AQZZQA
```

Рисунок 51 – Файл input.txt с входными данными из примера 3



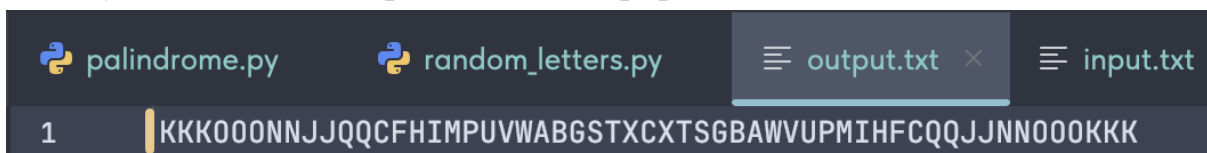
```
palindrome.py random_letters.py output.txt x input.txt
1 | 6
2 | ABCDEF
```

Рисунок 52 – Файл output.txt с входными данными из примера 3



```
palindrome.py random_letters.py output.txt x input.txt
1 | 70
2 | UYRDWBNAHTKPKKOVONKKXVCJMQAFOTFJMSIPQKVOOCNGWHGEIQJJXFBGUNNMQSLWONIKUP
```

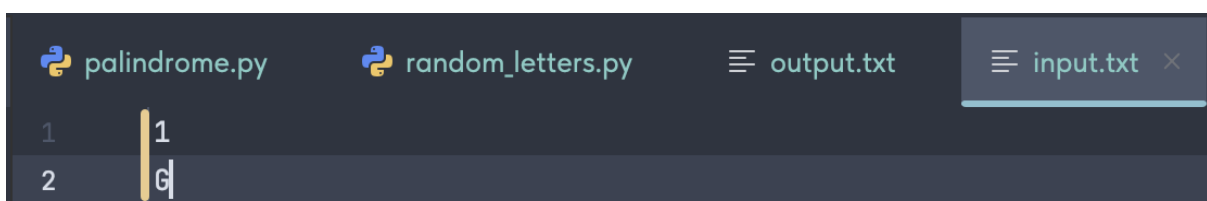
Рисунок 53 – Файл input.txt со сгенерированными входными данными



```
palindrome.py random_letters.py output.txt x input.txt
1 | KKK000NNJJQQCFHIMPUVWABGSTXCXTSGBAWVUPMIFCQQJJNN000KKK
```

Рисунок 54 – Результат работы кода для сгенерированных входных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:



```
palindrome.py random_letters.py output.txt x input.txt
1 | 1
2 | G
```

Рисунок 43 – Минимальное вводимое значение в input.txt

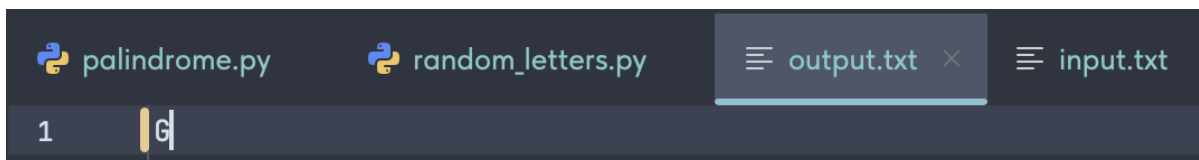


Рисунок 44 – Результат работы кода для значения $n = 1$, записанный в output.txt

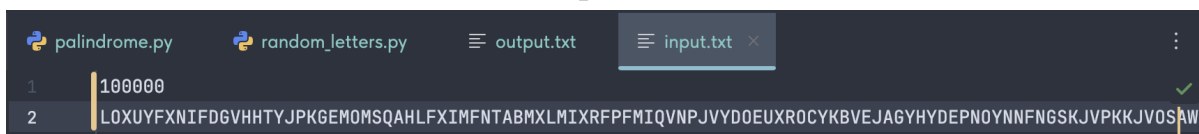


Рисунок 45 – Максимальное вводимое значение в input.txt

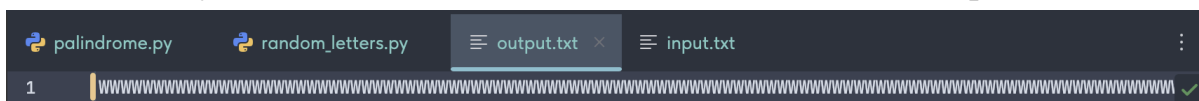


Рисунок 46 – Результат работы кода для значения $n = 100000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 10:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0006 сек	118 байт
Пример 1	0.0013 сек	162 байт
Пример 2	0.0014 сек	165 байт
Пример 3	0.0010 сек	118 байт
Сгенерированный пример	0.0012 сек	1054 байт
Верхняя граница диапазона значений входных данных из текста задачи	0.3792 сек	98.7 Кбайт

Вывод по задаче: В функции создания палиндрома были использованы алгоритмы сортировки вставкой по возрастанию и убыванию, линейного поиска. С помощью алгоритмов создание палиндрома из 100000 символов заняло не более 0.4 сек.

Вывод

Задачи сортировки разнообразны и далеко не всегда эффективны (как раз сортировка вставками). С помощью их реализации можно решить множество других интересных и нетривиальных задач.